

Software Requirements Specification (SRS)

Secure Personal Finance Tracker

Field	Details
Author	Sahibenoor Singh Sidhu
Course	Applied Computer Science, University of Winnipeg
Document	SRS v1.0
Date	June 26, 2026

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) formally defines the functional and non-functional requirements for the Secure Personal Finance Tracker, Version 1. It serves as the authoritative reference for design, implementation, and testing decisions throughout the project. The document ensures that every major feature planned in the project proposal maps to at least one verifiable requirement, and that the main user interactions are represented through use cases.

1.2 Scope

The Secure Personal Finance Tracker is a beginner level full-stack web application that allows individual users to manually record, manage, and review their personal income and expenses. Version 1 is intentionally scoped to a core set of features: secure authentication, transaction management (add, edit, delete, view), filtering, a monthly summary dashboard with a spending chart, and an on-demand monthly spending summary delivered to the user's registered email address via an external email API. It does not yet include bank integration, custom categories, budget goals, mobile app, recurring transactions, password reset, multi-currency support, multi-user sharing and advanced analytics etc.

1.3 Definitions and Acronyms

Term	Meaning
FR	Functional requirement
NFR	Non-Functional requirement
UC	Use Case
SQL	Structured Query Language
bcrypt	Password-hashing library used to store passwords securely
Transaction	A recorded income or expense entry.
CRUD	Create, Read, Update, and Delete.
Session	Server-side login state that keeps the user authenticated.

Dashboard	The summary page showing monthly totals and a spending chart
Email API	An external service used by the backend to send emails to users
API Key	A secret credential used to authenticate requests to the Email API; stored via environment variables, never in source code
email_logs	Database table recording each attempted monthly report email and its delivery status

1.4 Document Overview

Section 2 provides an overall product description. Section 3 lists all functional requirements. Section 4 lists non-functional requirements. Section 5 presents seven fully specified use cases covering the primary user interactions.

2. Overall Description

2.1 Product Perspective

The Secure Personal Finance Tracker is a standalone web application. It does not integrate with any external financial institution in Version 1. The system consists of three tiers: a browser-based frontend (HTML, CSS, JavaScript, Bootstrap, Chart.js), a Node.js/Express.js backend, and a PostgreSQL relational database. All transaction and user data resides locally on the server, no cloud storage or financial-institution APIs are used. The single external dependency is a third-party Email API, used exclusively to deliver the on-demand monthly spending summary to the user's registered email address. The API key for this service is stored using environment variables (dotenv) and is never committed to source control.

2.2 User Classes

The main user class is an individual budget tracker, such as a student, entry-level employee, or beginner budget-conscious person. The user is expected to want a simple manual tracker rather than advanced finance automation. Version 1 has no administrator role and no multi-user sharing role.

2.3 Assumptions and Dependencies

- The application will be deployed and tested on localhost for Version 1.
- Node.js (v18+), npm, and PostgreSQL are installed on the development machine.
- The developer will use Git and GitHub for version control throughout the project.
- All financial amounts are assumed to be in a single fixed currency (CAD) for V1.
- Password resets via email are not implemented in V1, resets require direct database access.
- A valid Email API account and API key are available and configured via environment variables before the monthly report feature can be tested.
- The user's registered email address (used at sign-up) is assumed to be a valid, deliverable address, the system does not verify email ownership in V1.

2.4 Constraints

Version 1 must stay within the locked first-project scope. The system must use fixed categories only. All sensitive user data must be protected through password hashing, secure session handling, user-specific database queries, server-side validation, and parameterized SQL queries. The application must remain understandable and maintainable for a first serious full-stack project.

3. Functional Requirements

Authentication

ID	Requirement
FR-01	The system shall allow a new user to register with an email address and password.
FR-02	The system shall validate registration input, including email format, minimum password length of 8 characters, and duplicate email checking.
FR-03	The system shall hash every user password before storing it in the database.
FR-04	The system shall allow a registered user to log in using email and password.
FR-05	The system shall reject login attempts with an incorrect email or password and display an error message.
FR-06	The system shall create a server-side session upon successful login to maintain the user's authenticated state.
FR-07	The system shall allow a logged-in user to log out, which destroys the current session.

Transaction Management

ID	Requirement
FR-08	The system shall allow a logged-in user to add a transaction by providing: amount, type (income or expense), category, date, and an optional note.
FR-09	The system shall validate that the amount field is a positive number greater than zero.
FR-10	The system shall restrict the category field to the fixed list: Food, Transport, Housing, Entertainment, Health, Salary, Other.
FR-11	The system shall allow a logged-in user to edit any field of a transaction they own.
FR-12	The system shall allow a logged-in user to delete a transaction they own.
FR-13	The system shall prevent a user from viewing, editing, or deleting transactions that belong to another user.

Transaction List and Filters

ID	Requirement
FR-14	The system shall display all of the logged-in user's transactions in a table, showing amount, type, category, date, and note.
FR-15	The system shall allow the user to filter the transaction list by month and year.
FR-16	The system shall allow the user to filter the transaction list by transaction type (income or expense).
FR-17	The system shall allow the user to filter the transaction list by category.
FR-18	The system shall display the filtered results without a full page reload (or with a standard form-based reload if JavaScript filtering is not implemented).

Dashboard

ID	Requirement
FR-19	The system shall display the total income for the current month on the dashboard.
FR-20	The system shall display the total expenses for the current month on the dashboard.
FR-21	The system shall display the net balance (income minus expenses) for the current month on the dashboard.
FR-22	The system shall display a bar chart or pie chart showing spending broken down by category for the current month.

Monthly Email Report

ID	Requirement
FR-23	The system shall provide a button allowing a logged-in user to request a monthly spending summary email for a selected month.
FR-24	The system shall calculate the selected month's total income, total expenses, net balance, and spending by category before composing the email.
FR-25	The system shall send the composed summary to the email address registered to the logged-in user's account, via the configured Email API.
FR-26	The system shall record each email send attempt in the email_logs table, including recipient address, email type, status (success or failure), and timestamp.
FR-27	The system shall display a confirmation message to the user when the email is sent successfully, and an error message if delivery fails.
FR-28	The system shall not expose the Email API key or any other transactional credential to the frontend or client-visible code.

4. Non-Functional Requirements

ID / Type	Requirement
NFR-01 Security	The system shall store passwords only as bcrypt hashes and never as plaintext.
NFR-02 Security	The system shall use parameterized SQL queries for all database operations to reduce SQL injection risk.
NFR-03 Security	The system shall enforce user data isolation so that one user cannot view, edit, or delete another user's transactions.
NFR-04 Security	The system shall use server-side sessions using express-session to authenticate requests, no sensitive financial data, passwords, or API keys shall be stored client-side.
NFR-05 Security	The system shall store the Email API key exclusively in environment variables (via dotenv) and shall never log, expose, or commit the key to source control.
NFR-06 Usability	The system shall show clear validation and error messages when a form submission fails.
NFR-07 Usability	The system shall use consistent Bootstrap styling for forms, buttons, navigation, tables, and dashboard cards.
NFR-08 Performance	The system shall load major pages, including the dashboard and transaction list, in under 2 seconds on localhost with normal sample data.
NFR-09 Performance	Database queries shall include appropriate indexes on user_id and transaction_date columns to avoid full-table scans
NFR-10 Maintainability	The system shall keep backend code organized into clear modules such as routes, controllers, database connection, middleware, and views/static files.

5. Use Cases

The following use cases cover the main Version 1 features and will later be used to create manual test cases.

UC-01 Register Account

Field	Details
Use Case ID	UC-01
Use Case Name	Register Account
Actor	Unregistered Visitor
Precondition	The user has navigated to the registration page and does not already have an account.
Main Flow	1. User enters a valid email address and a password in the registration form. 2. User submits the form. 3. System validates that the email is not already registered. 4. System hashes the password using bcrypt. 5. System stores the new user record (email, password_hash, created_at) in the users table. 6. System redirects the user to the login page with a success message.
Alternate Flow	Invalid email, weak password, or duplicate email shows an error and the account is not created.
Postcondition	A new user record exists in the database. The user can now log in.
Related FRs	FR-01, FR-02, FR-03

UC-02 Log In

Field	Details
Use Case ID	UC-02
Use Case Name	Log in
Actor	Registered user
Precondition	The user has an existing account and is not logged in.
Main Flow	1. User enters their email and password on the login page. 2. User submits the form. 3. System retrieves the user record matching the email. 4. System compares the submitted password with the stored bcrypt hash. 5. System creates a server-side session storing the user's ID. 6. System redirects the user to the dashboard.
Alternate Flow	Wrong password or unknown email shows an error and the session is not created.
Postcondition	The user is authenticated; a session exists. They are on the dashboard page
Related FRs	FR-04, FR-05, FR-06

UC-03 Add Transaction

Field	Details
Use Case ID	UC-03
Use Case Name	Add Transaction
Actor	Logged-in user
Precondition	The user is authenticated and logged in.
Main Flow	1. Open the Add Transaction page. 2. Enter amount, type, category, date, and optional note. 3. Submit the form. 4. The system validates input. 5. The system saves the transaction with the current user ID. 6. The system redirects to the transaction list.
Alternate Flow	Missing or invalid fields show an error and no transaction is saved.
Postcondition	A new transaction is stored for the logged-in user.
Related FRs	FR-08, FR-09, FR-10

UC-04 View Dashboard

Field	Details
Use Case ID	UC-04
Use Case Name	View Dashboard
Actor	Logged-in user
Precondition	The user is logged in and may or may not have transactions.
Main Flow	1. Open the dashboard. 2. The system retrieves current-month transactions for the user. 3. The system calculates income, expenses, and net balance. 4. The system displays totals and category chart.
Alternate Flow	If no current-month data exists, the system shows a helpful empty message instead of a broken chart.
Postcondition	The user sees an accurate monthly financial summary.
Related FRs	FR-19, FR-20, FR-21, FR-22

UC-05 Delete Transaction

Field	Details
Use Case ID	UC-05
Use Case Name	Delete Transaction
Actor	Logged-in user
Precondition	The user is logged in and owns at least one transaction.
Main Flow	1. Open the transaction list. 2. Click delete on an owned transaction. 3. Confirm deletion. 4. The system checks that the transaction belongs to the current user. 5. The system deletes the record. 6. The system returns to the list.
Alternate Flow	If the user cancels, nothing is deleted. If the transaction does not belong to the user, the system refuses the action.
Postcondition	The selected owned transaction is removed from the user's list.
Related FRs	FR-12, FR-13

UC-06 Send Monthly Email Report

Field	Details
Use Case ID	UC-06
Use Case Name	Send Monthly Email Report
Actor	Logged-in user
Precondition	The user is logged in and has a registered email address.
Main Flow	1. The user opens the dashboard page. 2. The user selects or views the monthly summary. 3. The user clicks the button to send the monthly spending summary by email. 4. The system calculates total income, total expenses, net balance, and spending by category for the selected month. 5. The system sends the summary to the user's registered email address using the Email API. 6. The system displays a success message.
Alternate Flow	If the Email API fails or the email cannot be sent, the system displays a clear error message and does not crash.
Postcondition	The monthly spending summary is sent to the user's registered email address, or the user is informed that the email could not be sent.
Related FRs	FR-23, FR-24, FR-25, FR-26, FR-27, FR-28

UC-07 Filter Transactions

Field	Details
Use Case ID	UC-07
Use Case Name	Filter Transactions
Actor	Logged-in user
Precondition	The user is logged in and has access to the transaction list page.
Main Flow	1. The user opens the transaction list page. 2. The system displays the user's transactions in a table. 3. The user selects one or more filters, such as month and year, transaction type, or category. 4. The user applies the selected filters. 5. The system retrieves and displays only the transactions that match the selected filter criteria.
Alternate Flow	If no transactions match the selected filters, the system displays an empty result message instead of showing a blank or broken table.
Postcondition	The user sees a filtered list of transactions based on the selected criteria.
Related FRs	FR-14, FR-15, FR-16, FR-17, FR-18

6. Requirement Coverage Review

Proposal Feature	Covered By
Authentication	FR-01–07; NFR-01, NFR-04; UC-01, UC-02
Transactions CRUD	FR-08–14; UC-03, UC-05
Fixed categories	FR-10; UC-03
Transaction filters	FR-15–18; UC-07
Dashboard totals and chart	FR-19–22; UC-04
Monthly email report	FR-23–28; NFR-05, NFR-10; UC-06
Security and data isolation	FR-13, FR-28; NFR-01–05; UC-05
Testing and documentation support	NFR-06, NFR-07, NFR-10; UC-01–07